# Reviewer Pack — Minimal Rank of Free Entropy over Graphs vs Distinguishing T…

Entry a0e60f6945e2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 07:20:49 UTC

## Minimal Rank of Free Entropy over Graphs vs Distinguishing Tensor Width for BP_ReadTwice

**Entry ID**: `a0e60f6945e2` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 07:20:49 UTC

### 1. Conjecture

**Field A** (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs

**Statement**:

> ['The distinguishing tensor width (DTW) of a read-twice branching program is lower-bounded by the free entropy of the associated graph, i.e., DTW(BP) ≥ F(G), where BP is a read-twice branching program on n inputs and G is its associated graph.', 'For all read-twice branching programs BP with input size n, there exists a function f(n) such that the ratio of the free entropy of the associated graph, F(G), to DTW(BP) satisfies F(G)/DTW(BP) ≥ f(n).', 'The quantity F(G) provides a tight lower bound on the tensor width for read-twice branching programs.']

**Rationale (proposer's reasoning)**:

> ['Free probability theory provides a framework that captures the probabilistic structure of graphs, which is relevant to understanding the complexity of computations. The free entropy has been used in other contexts to analyze the complexity of certain problems, suggesting its potential utility in this setting.', 'Read-twice branching programs are a class of branching programs with strong connections to communication complexity and circuit complexity. By linking free entropy to read-twice branching programs, we may uncover new insights into the inherent complexity of these problems.', 'The conjecture proposes a direct connection between two seemingly unrelated fields, potentially revealing underlying structures that can be exploited to design more efficient algorithms or prove lower bounds.']

**Taxonomy category**: `COMM_DISJ` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `4621b0947cbc5549`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

For all read-twice branching programs BP with input size n ≤ 40, if the ratio of the free entropy F(G) to the distinguishing tensor width DTW(BP) for each instance is greater than or equal to a function f(n), then support the conjecture. If any instance has F(G)/DTW(BP) < f(n), falsify the conjecture.

## 3. Barrier filter (F1)

**Final**: PASS

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 1.00 | UNCERTAIN | SAFE |

## 4. Novelty audit

**Verdict**: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "free entropy" AND "distinguishing tensor width" AND "read-twice branching programs" - "graph associated free entropy" AND "tensor width" AND "BP_ReadTwice" - "lower bound" free entropy "tensor width" "read-twice branching programs"

**Top relevant hits considered**: - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem - [http://arxiv.org/abs/2511.07739v3] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [http://arxiv.org/abs/1201.0716v2] Concavification of free entropy

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        n = len(matrix)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            factor = matrix[i][i]
            for j in range(n):
                matrix[i][j] /= factor
            for j in range(n):
                if j != i:
```

```
                factor = matrix[j][i]
                for k in range(n):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix

def matrix_multiply(A, B):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    det = 0
    sign = 1
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in matrix[1:]]
        det += sign * matrix[0][i] * determinant(submatrix)
        sign *= -1
    return det

def free_entropy(graph):
    n = len(graph)
    laplacian = [[0]*n for _ in range(n)]
    for i in range(n):
        degree = sum(graph[i])
        laplacian[i][i] = degree
        for j in range(i+1, n):
            if graph[i][j]:
                laplacian[i][j] = -1
                laplacian[j][i] = -1

    eigenvalues = []
    for _ in range(10):  # Power iteration method to find the largest eigenvalue
        v = [random.random() for _ in range(n)]
        v /= math.sqrt(sum(x**2 for x in v))
        for _ in range(10):
            v = matrix_multiply(laplacian, v)
            v /= math.sqrt(sum(x**2 for x in v))
        eigenvalues.append(v[0])

    return -sum(math.log(eigenvalue) for eigenvalue in eigenvalues if eigenvalue > 0)

def tensor_width(bp):
    # Placeholder function to calculate the tensor width
    # This is a dummy implementation and should be replaced with actual logic
    return len(bp)

n = random.randint(5, 40)
bp = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
G = construct_graph(bp)
```

```
    F_G = free_entropy(G)
    DTW_BP = tensor_width(bp)

    if DTW_BP == 0:
        return {
            "metric_name": "F(G)/DTW(BP)",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "tensor_width_is_zero"
        }

    ratio = F_G / DTW_BP

    return {
        "metric_name": "F(G)/DTW(BP)",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": ratio >= f(n),
        "counterexample": ""
    }

def construct_graph(bp):
    n = len(bp)
    G = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if bp[i][j]:
                G[i][j] = 1
                G[j][i] = 1
    return G

def f(n):
    # Placeholder function to define the lower bound f(n)
    # This is a dummy implementation and should be replaced with actual logic
    return n / 2

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [37, 41, 43, 47, 53, 59, 61,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_ratio = sum(result["metric_value"] for result in results if "metric_value" in result)
    mean_ratio = total_ratio / len(results) if results else 0
    std_ratio = math.sqrt(sum((result["metric_value"] - mean_ratio)**2 for result in results if "m

    support_fraction = sum(1 for result in results if "conjecture_holds" and result["conjecture_ho

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_frac
    elif any(not result["conjecture_holds"] for result in results):
```

```
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"f(n) is too small\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f998a59.py", line 133, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f998a59.py", line 91, in run_trial
    F_G = free_entropy(G)
          ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f998a59.py", line 74, in free_entropy
    v /= math.sqrt(sum(x**2 for x in v))
TypeError: unsupported operand type(s) for /=: 'list' and 'float'
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> The test code crashed during execution, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the error in the test code to ensure it can complete without crashing.

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 16651 |
| 2 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 6002 |
| 3 | novelty | ollama_remote | glm4:latest | 0 | 0 | 4774 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 6437 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 13885 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 12755 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 15068 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 15909 |
| 9 | judge | ollama_remote | glm4:latest | 0 | 0 | 7905 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 99386 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/a0e60f6945e2.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a0e60f6945e2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/a0e60f6945e2.tar.gz` (if generated)